Disjoint Set Union

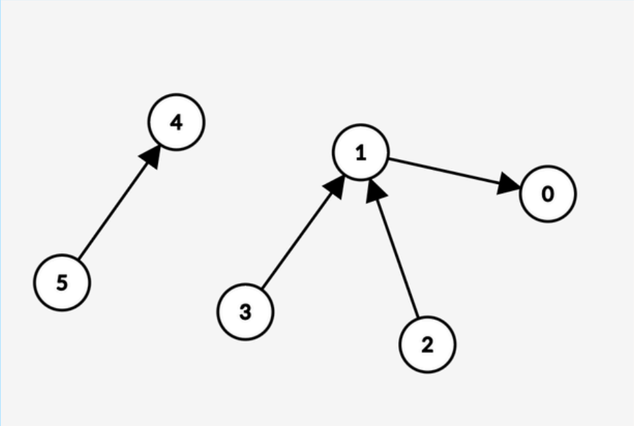
SHSID CPC

What is DSU/Union Find?

Stores and merges sets of values as trees

Two functions:

1. Find(node): returns the “index” of the set containing the node
2. Union(node1, node2): merges the sets containing node1 and node2.



Two sets: {4,5} and {0,1,2,3}. Note that the edges are directed (direction will be useful).

“Basic” (not optimized) DSU

Find: go up the node’s tree (to its ancestors) until we reach a root node.

```
def Find(node):
    if father(node) == None:
        return node
    return Find(father(node))
```

Union: Find the roots of the nodes and set one of the roots as the other’s parent

```
def Union(node1, node2):
    root1 = Find(node1)
    root2 = Find(node2)
    if root1 != root2:
        father(root1) = root2
```

Time complexity: O(mlog(n)) on average, O(n*m) worst case if we do this m times

“Better” (optimized) DSU

During the Find function, the node travels up the tree so that further accessing will be O(1) (Union remains unchanged)

Find: We connect the node directly to the root, so we “compress” the tree, making future Find functions much faster as we only need to access 2 elements in future Finds

```
def Find(node):
    if father(node) == None:
        return node
    return father(node) = Find(father(node))
```

Time complexity: O(log(n)+m) on average, O(n+m) worst case if we do this m times

```cpp
// initialized:
// fa[i] = i, sz[i] = 1
const int MAXN = 1005;
int fa[MAXN];
int sz[MAXN];

int find(int x){
    if (fa[x] == x) return x;
    else fa[x] = find(fa[x]);
}

int Union(int x, int y){
    x = find(x);
    y = find(y);
    fa[x] = y;
    sz[x] += sz[y];
}

int UnionOpt(int x, int y){
    x = find(x);
    y = find(y);
    if (sz[x] > sz[y]) swap(x, y);
    fa[x] = y;
    sz[x] += sz[y];
}
```

USACO 2020 Jan Silver P3: Wormhole Sort (again)

USACO 2020 January Contest, Silver
Problem 3. Wormhole Sort

Return to Problem List

Contest has ended.

Submitted; Results below show the outcome for each judge test case

1	2	3	4	5	6	7	8	9	10
*	*	*	*	*	*	*	*	*	*
3.4mb 2ms	3.4mb 2ms	3.4mb 2ms	3.4mb 2ms	3.4mb 2ms	4.6mb 55ms	4.7mb 71ms	4.8mb 61ms	4.9mb 57ms	4.9mb 86ms

English (en)

Farmer John's cows have grown tired of his daily request that they sort themselves before leaving the barn each morning. They have just completed their PhDs in quantum physics, and are ready to speed things up a bit.

This morning, as usual, Farmer John's N cows ($1 \le N \le 10^5$), conveniently numbered $1 \ldots N$, are scattered throughout the barn at N distinct locations, also numbered $1 \ldots N$, such that cow i is at location p_i. But this morning there are also M wormholes ($1 \le M \le 10^5$), numbered $1 \ldots M$, where wormhole i bidirectionally connects location a_i with location b_i, and has a width w_i ($1 \le a_i, b_i \le N, a_i \ne b_i, 1 \le w_i \le 10^9$).

At any point in time, two cows located at opposite ends of a wormhole may choose to simultaneously swap places through the wormhole. The cows must perform such swaps until cow i is at location i for $1 \le i \le N$.

The cows are not eager to get squished by the wormholes. Help them maximize the width of the least wide wormhole which they must use to sort themselves. It is guaranteed that it is possible for the cows to sort themselves.

SCORING:

- Test cases 3-5 satisfy $N, M \le 1000$.
- Test cases 6-10 satisfy no additional constraints.

INPUT FORMAT (file wormsort.in):

The first line contains two integers, N and M.

The second line contains the N integers $p_1, p_2, \ldots, p_N$. It is guaranteed that p is a permutation of $1 \ldots N$.

For each i between 1 and M, line $i+2$ contains the integers a_i, b_i, and w_i.

OUTPUT FORMAT (file wormsort.out):

A single integer: the maximum minimal wormhole width which a cow must squish itself into during the sorting process. If the cows do not need any wormholes to sort themselves, output -1.

SAMPLE INPUT:

```
4 4
3 2 1 4
1 2 9
1 3 7
2 3 10
2 4 3
```

SAMPLE OUTPUT:

```
9
```

Here is one possible way to sort the cows using only wormholes of width at least 9:

- Cow 1 and cow 2 swap positions using the third wormhole.
- Cow 1 and cow 3 swap positions using the first wormhole.
- Cow 2 and cow 3 swap positions using the third wormhole.

SAMPLE INPUT:

```
4 1
1 2 3 4
4 2 13
```

SAMPLE OUTPUT:

```
-1
```

No wormholes are needed to sort the cows.

Problem credits: Dhruv Rohatgi

USACO 2020 Jan Silver P3: Wormhole Sort (again)

BS, as usual

BUT, each time, we do a DSU instead of a DFS/BFS on the graph, and check for connectivity

Time complexity: approximately O(mlogn)

USACO 2020 Jan Silver P3: Wormhole Sort (again)

```cpp
#include <bits/stdc++.h>
using namespace std;
struct wormhole {
    int a,b,w;
    bool operator < (const wormhole & other) const{
        return w < other.w;
    }
};
wormhole holes[100006];
int root[100005],n,m,idx[100005];
int Find(int a){
    if(root[a]==a) return a;
    return root[a] = Find( a: root[a]);
}
void Union(int a, int b) {
    root[Find(a)] = Find( a: b);
}
int main() {
    ios::sync_with_stdio( sync: false);
    cin.tie( tiestr: nullptr);
    freopen("wormsort.in","r",stdin);
    freopen("wormsort.out","w",stdout);
    cin >> n >> m;
    bool b = false;
    for(int i = 1; i <= n; i++){
        int a;
        cin >> a;
        idx[a] = i;
        if(idx[i]!=i) b = true;
    }
    for(int i = 1; i <= m; i++) cin >> holes[i].a >> holes[i].b >> holes[i].w;
    if(!b){
        cout << -1;
        return 0;
    }
    sort( first: holes+1, last: holes+m+1);
    int lo = 1, hi = 100003,mid;
    while(lo+1 < hi){
        for(int i = 1; i <= n; i++) root[i] = i;
        mid = (hi+lo)/2;
        for(int i = mid; i <= m; i++) Union(holes[i].a,holes[i].b);
        b = false;
        for(int i = 1; i <= n; i++){
            if(Find( a: i) != Find( a: idx[i])){
                hi = mid;
                b = true;
                break;
            }
        }
        if(!b) lo = mid;
    }
    cout << holes[lo].w;
    return 0;
}
```

USACO 2018 January Contest, Gold
Problem 1. MooTube

Return to Problem List

Contest has ended.

Log in to allow submissions in analysis mode

English (en)

In his spare time, Farmer John has created a new video-sharing service, which he names MooTube. On MooTube, Farmer John's cows can record, share, and discover many amusing videos. His cows already have posted N videos ($1 \le N \le 100,000$), conveniently numbered $1 \ldots N$. However, FJ can't quite figure out how to help his cows find new videos they might like.

FJ wants to create a list of "suggested videos" for every MooTube video. This way, cows will be recommended the videos most relevant to the ones they already watch.

FJ devises a metric of "relevance," which determines, as the name suggests, how relevant two videos are to each other. He picks $N - 1$ pairs of videos and manually computes their pairwise relevance. Then, FJ visualizes his videos as a network, where each video is a node and the $N - 1$ pairs of videos he manually considered are connected. Conveniently, FJ has picked his $N - 1$ pairs so that any video can be reached from any other video along a path of connections in exactly one way. FJ decides that the relevance of any pair of videos should be defined as the minimum relevance of any connection along this path.

Farmer John wants to pick a value K so that next to any given MooTube video, all other videos with relevance at least K to that video will be suggested. However, FJ is worried that too many videos will be suggested to his cows, which could distract them from milk production! Therefore, he wants to carefully set an appropriate value of K. Farmer John would like your help answering a number of questions about the suggested videos for certain values of K.

INPUT FORMAT (file mootube.in):

The first line of input contains N and Q ($1 \le Q \le 100,000$).

The next $N - 1$ lines each describe a pair of videos FJ manually compares. Each line includes three integers p_i, q_i, and r_i ($1 \le p_i, q_i \le N, 1 \le r_i \le 1,000,000,000$), indicating that videos p_i and q_i are connected with relevance r_i.

The next Q lines describe Farmer John's Q questions. Each line contains two integers, k_i and v_i ($1 \le k_i \le 1,000,000,000, 1 \le v_i \le N$), indicating that FJ's ith question asks how many videos will be suggested to viewers of video v_i if $K = k_i$.

OUTPUT FORMAT (file mootube.out):

Output Q lines. On line i, output the answer to FJ's ith question.

USACO 2025 January Contest, Gold
Problem 2. Reachable Pairs

Return to Problem List

Contest has ended.

Log in to allow submissions in analysis mode

English (en)

Consider an undirected graph with N nodes labeled $1 \ldots N$ and M edges ($1 \le N \le 2 \cdot 10^5, 0 \le M \le 4 \cdot 10^5$). You're given a binary string $s_1 s_2 \ldots s_N$. At time t for each $t \in [1, N]$,

- If $s_t = 0$, node t is removed from the graph.
- If $s_t = 1$, node t is removed from the graph, and edges are added between every pair of neighbors that node t had just before removal.

Note that in both cases, when a node is removed from the graph all of its incident edges are removed as well.

Count the number of pairs of nodes that can reach each other via some sequence of edges just before each of timesteps $1 \ldots N$.

INPUT FORMAT (input arrives from the terminal / stdin):

The first line contains N and M.

The second line contains the bit string s of length N.

The next M lines each contain two integers denoting an edge of the graph.

OUTPUT FORMAT (print output to the terminal / stdout):

N lines, the number of pairs before each timestep.

SAMPLE INPUT:

```
3 2
111
1 2
1 3
```

SAMPLE OUTPUT:

```
3
1
0
```

Reverse the process, everyone is isolated

if 0: connect to all neighbors
if 1: don't move
connect all ones to neighbors beforehand

keep track of size